



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
INSTITUTO METRÓPOLE DIGITAL
PROCESSOS DE SOFTWARE

Ana Beatriz Camilo da Costa
Arthur Boma Skeete Mypoyo
Fernando Silva dos Santos
Francisca Gabrielly Lopes Freire

SISTEMA GERENCIADOR DE APRESENTAÇÕES MUSICAIS

2026
Natal

1. Introdução.....	2
2. Visão do produto.....	2
3. Definição do MVP.....	2
Dentro do escopo (MVP):.....	2
Fora do escopo (MVP):.....	3
4. Product backlog inicial.....	3
5. Stack tecnológico.....	4
6. Acordo de processo.....	4
7. Equipe.....	5
8. Links Importantes.....	5

1. Introdução

Na Escola de Música da Universidade Federal do Rio Grande do Norte (UFRN), há uma ampla variedade de grupos musicais que frequentemente se apresentam em eventos acadêmicos da universidade. Atualmente, o processo de solicitação para apresentações é gerenciado pela secretaria de extensão da escola. O procedimento consiste em o solicitante entrar em contato por e-mail com a secretaria que, em seguida, registra manualmente a solicitação em uma tabela e verifica a disponibilidade dos grupos musicais para atender ao pedido. Além disso, após a confirmação da disponibilidade do grupo, é necessário que o solicitante providencie e confirme o transporte. Toda essa comunicação é feita de forma manual, o que implica em possíveis erros. Com isso em mente, surgiu a ideia de desenvolver um projeto que automatize todo o processo, permitindo que ambas as partes acompanhem a solicitação e o agendamento de forma eficiente. Essa solução visa facilitar a comunicação entre o solicitante e o grupo musical, além de minimizar possíveis erros de logística e falhas de comunicação.

2. Visão do produto

- Para grupos musicais, organizadores de eventos e instituições culturais/acadêmicas
- Que enfrentam dificuldades no controle logístico, gestão de integrantes, alocação de instrumentos e organização de deslocamentos para apresentações
- O Sistema Gerenciador de Apresentações Musicais é um software de gestão e logística web
- Que centraliza e automatiza a administração de membros, controle de eventos, designação de cargos e solicitações de transporte
- Diferente de controles manuais feitos em planilhas eletrônicas ou sistemas de gestão genéricos que não compreendem a realidade de um grupo artístico
- Nosso produto integra, em uma única plataforma, a gestão de pessoal (músicos e instrumentos) com a complexa logística de infraestrutura e transporte necessária para a execução de eventos musicais.

3. Definição do MVP

O objetivo inicial do MVP é validar o fluxo central de marcação de um evento e a alocação de recursos (pessoas e transporte) para que ele aconteça. Inicialmente, iremos implementar apenas uma api para validar a ideia, se houver tempo hábil implementaremos o frontend.

Dentro do escopo (MVP):

- Autenticação e autorização básica com perfis de usuário (aluno, funcionário, administrador).
- Cadastro e gerenciamento de grupos musicais e seus respectivos membros.
- Registro e vinculação de instrumentos musicais aos membros.
- Criação de eventos (data, local, descrição e responsáveis).
- Módulo de solicitação de apresentação (vincular um grupo a um evento).

- Módulo de solicitação de transporte (cadastro básico de veículos, motoristas e rotas para um evento).

Fora do escopo (MVP):

- Gestão financeira (pagamentos de cachês, emissão de notas ou venda de ingressos).
- Notificações em tempo real (Push notifications ou chat interno).
- Integração com APIs externas de mapas (Google Maps/Waze) para cálculo de rotas dinâmicas.
- Controle de estoque de manutenção de instrumentos.

4. Product backlog inicial

Como o foco da disciplina está nos processos, o backlog mescla histórias de negócio com tarefas técnicas fundamentais para a arquitetura:

1. **Task técnica:** Configurar a estrutura inicial do projeto Spring Boot, conexão com o banco PostgreSQL e estruturação do repositório no GitHub.
2. **Task técnica:** Implementar a esteira de CI no GitHub Actions configurada para rodar o *build* do Maven e testes a cada *Push/Pull Request* na branch principal.
3. **Task técnica:** Integrar e configurar o Swagger/OpenAPI para geração automática da documentação dos endpoints.
4. **US01:** Como administrador, quero cadastrar e gerenciar perfis de usuários (alunos e funcionários) para garantir que apenas pessoas autorizadas acessem o sistema.
5. **US02:** Como coordenador, quero criar um grupo musical e adicionar integrantes a ele para ter o controle de quem participa de cada projeto.
6. **US03:** Como músico/coordenador, quero registrar instrumentos musicais e vinculá-los aos integrantes para saber os recursos técnicos disponíveis.
7. **US04:** Como produtor, quero cadastrar um evento informando local, data e responsáveis para organizar a agenda do grupo.
8. **US05:** Como produtor, quero criar uma solicitação de apresentação vinculando um grupo a um evento aprovado.
9. **US06:** Como responsável logístico, quero registrar veículos e motoristas para criar Solicitações de Transporte associadas aos eventos agendados.

5. Stack tecnológico

A arquitetura foi definida para priorizar a qualidade do processo de desenvolvimento, a documentação e a robustez do back-end:

- **Back-end:** Java com Spring Boot.
 - *Módulos:* Spring Web (para os controladores REST), Spring Data JPA (para abstração do acesso a dados e mapeamento objeto-relacional).
 - *Justificativa:* Ecossistema robusto, amplamente utilizado no mercado corporativo, que acelera o desenvolvimento de APIs e facilita a implementação de testes unitários e de integração. Além disso, todos os membros da equipe têm domínio da tecnologia.
- **Banco de Dados:** PostgreSQL.
 - *Justificativa:* SGBD relacional de código aberto, altamente confiável e que se integra perfeitamente ao ecossistema Spring/JPA para garantir a integridade dos dados de logística e pessoal.
- **Documentação de API:** Swagger / Springdoc OpenAPI.
 - *Justificativa:* Essencial para projetos *API-first* ou sem front-end inicial. Permite que desenvolvedores testem os endpoints diretamente pelo navegador e gera uma documentação viva e padronizada.
- **DevOps e Processos:** GitHub Actions.
 - *Justificativa:* Ferramenta nativa do repositório, ideal para criar a pipeline de Integração Contínua (CI), garantindo que nenhum código que quebre o build ou os testes seja mesclado à branch principal.

6. Acordo de processo

- **Metodologia:** Ágil (Scrum/Kanban adaptado).
- **Cadência de Sprints:** 2 semanas.
- **Gestão de Código:** Git Flow adaptado (branches de feature partindo da develop, abrindo Pull Requests para code review).
 - **Pipeline de CI (GitHub Actions):** A cada Push ou abertura de Pull Request para a branch develop, uma esteira automatizada será engatilhada para realizar o build da aplicação e executar a suíte de testes. A mesclagem do código só é permitida se a pipeline passar sem erros.
- **Cerimônias:**
 - *Planning:* Definição das histórias do backlog que entrarão na Sprint.
 - *Daily:* Acompanhamento assíncrono (via whatsapp).
 - *Review:* Reunião ao final da Sprint para demonstrar as funcionalidades concluídas (endpoints funcionando e documentados) e garantir o alinhamento com os requisitos.
 - *Retrospective:* Reunião após a *Review* para avaliar o que funcionou bem, o que falhou no fluxo de trabalho e propor melhorias para a próxima Sprint.
- **Ferramentas de Gestão e DevOps:** Quadro Kanban (GitHub Projects), versionamento de código (Git) e automação de rotinas (GitHub Actions).

- **Definition of Done (DoD):**

- O código foi implementado e está versionado no repositório.
- O padrão arquitetural (MVC / API RESTful) e de nomenclatura do projeto foi respeitado.
- A funcionalidade atende aos critérios de aceite da *User Story* (testada manualmente via Postman/Swagger e com testes de unidade e de integração).
- A pipeline de Integração Contínua (CI) no GitHub Actions executou com sucesso (Build e Testes aprovados).
- O código passou por revisão de outro membro (Pull Request aprovado).

7. Equipe

- Ana Beatriz Camilo da Costa - Desenvolvedora e Scrum Master
- Arthur Boma Skeete Mypoyo - Desenvolvedor full s
- Fernando Silva dos Santos - Analista de Sistema e Desenvolvedor
- Francisca Gabrielly Lopes Freire - Desenvolvedora e Devops

8. Links Importantes

- **Repositório de código:** <https://github.com/gabrielly-freire/sgam-api>
- **Quadro de tarefas (Kanban):** <https://github.com/users/gabrielly-freire/projects/5>